

Crucible — Engineering Scope of Work

Direction: ML + LLM hybrid · **Showcase:** June 29, 2026 (10-min live demo) · **Team:** clean four-way split

One line. An adversarial security platform where an LLM red agent attacks an AI target, an ensemble of independent oracles catches the cheats *without ever seeing the submission*, a blue LLM loop auto-hardens the target, and the whole rig grades its own catch rate against an adversary that already knows the verification scheme.

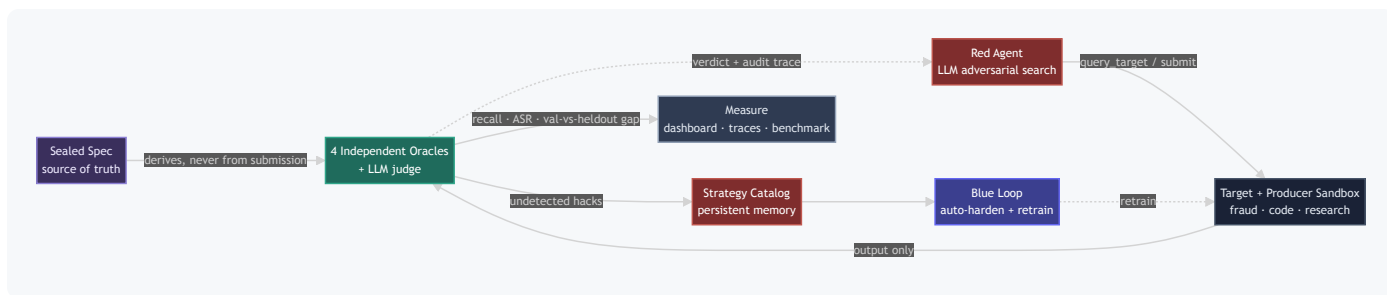
Provenance. Architecture derived solely from the project proposal. Proposal-stage: no implementation exists yet — dashboard metrics below populate from real runs, never from placeholders.

1. Scope of work (the honest minimum we commit to ship)

- **Spec sealing + producer sandbox** — the target runs with no path to the verification artifacts. This independence is the foundation everything else rests on.
- **Four independent oracles in parallel** over one real domain (code), plus a fraud-model adapter so the red/blue loop demos visually: (1) held-out tests generated *after* submission, never exposed; (2) metamorphic relations (no reference answer needed); (3) differential oracle (second implementation from a different model family must agree); (4) property-based fuzzing on invariants. An LLM judge is one measured vote, never the sole authority.
- **Red agent** finds ≥ 3 distinct evasion / reward-hack strategies with a measurable attack-success-rate, plus a **white-box adversary** handed the full scheme and rewarded for cheating anyway → turns “we catch cheats” into a measured recall number.
- **One blue-hardening round** that visibly recovers detection rate, verified on held-out attacks.
- **Measurement dashboard + audit traces + exported seeded-hack corpus and leaderboard** as a reusable benchmark.

Stretch (only if core lands early): time-series anomaly target · red/blue co-evolution over N rounds with a convergence curve · spec compiler · learned verifier-router tournament + human-review-budget meter.

2. Architecture diagram



Core loop: RED (attack) → VERIFY (independent oracles) → HARDEN (blue loop) → MEASURE (self-test) → re-attack. Dashed edges are feedback. Target-agnostic: one core, three adapters.

Trust boundary (the core bet). The producer **can see** its own input, its own output, and the `query_target()` score. It **cannot see** held-out test instances, the differential second implementation, or any oracle internals. White-box mode reveals the *scheme*, not the *instances* — converting “we catch cheats” into a measured recall number.

3. Why this is a huge innovation

Most “AI safety” demos ship a polished dashboard over a check the producer can quietly game — a circular argument dressed up as a product. Crucible inverts that. The hard, unsolved core is **independent verification that survives a white-box adversary**: checks that establish ground truth *without trusting the thing being checked*, that still hold when the producer knows exactly how it’s being checked.

Three things make it genuinely new:

1. **LLM semantic reasoning replaces gradients as the search engine** — finding evasions that are *valid and interpretable* (actionable blind spots), not real-world-illegal noise.
2. **A non-colluding oracle ensemble over a sealed spec** — four mechanisms that fail differently, plus a white-box adversary rewarded precisely for finding a shared gap.
3. **It grades itself** — continuously red-teams its own verifier and reports, as one number, how often a cheat still gets through, and halts certification when the residual crosses a red line.

Component for component, Crucible is a working testbed for problems on Anthropic’s published safety-research list: scalable oversight under exploitable oversight error, AI control / actor–monitor collusion, anomaly detection between models, adaptive defenses, auditable externalized reasoning, and RSP-style capability gating. The deliverable isn’t a claim that a two-week capstone solves these — it’s a **measured, reusable artifact pointed straight at the questions a frontier safety lab is actively asking**.

The pitch: we build an AI that breaks a system and then heals it — red team and blue team both LLM-driven — and the whole rig grades its own catch rate against an adversary that already knows the scheme. It lands on four hot themes at once: agents, adversarial robustness, scalable oversight, and AI safety.

Crucible · Gauntlet capstone proposal · architecture derived solely from the uploaded proposal. Full diagram set (8 Mermaid diagrams incl. end-to-end sequence + trust-boundary) lives in the repo README at github.com/scott-lydon/crucible.